
Tutorial 3

Computer Security BCS2420

Part A

1. Which of the following statements best reflects the difference between *online* and *offline* password guessing attacks?
 - ☐ In offline attacks, the attacker must query the legitimate server.
 - ☐ In online attacks, the attacker independently tests password guesses without interacting with a server.
 - ☐ In offline attacks, the attacker can test millions of password guesses per second without server interaction.
 - ☐ Online attacks generally allow for more guesses than offline attacks.
2. Which approach best prevents a *pre-computed dictionary* (*rainbow table*) attack on a password hash?
 - ☐ Storing passwords in cleartext files
 - ☐ Including salts in password hashing
 - ☐ Using plaintext-based challenge-response methods
 - ☐ Employing rate-limiting on user login attempts
3. A password-based user authentication system enforces *password stretching* by applying a hash 100,000 times to each password during verification. This primarily helps to:
 - ☐ Increase user convenience by speeding up verification
 - ☐ Prevent user lockouts
 - ☐ Slow down offline brute-force attacks
 - ☐ Make online brute-force attacks less frequent
4. Which factor is *not* a major contributor to an increased *False Accept Rate (FAR)* in a biometric system?
 - ☐ Loosening the matching threshold

- ☐ Highly distinctive biometric features among the population
 - ☐ Poor sensor quality causing noisy data
 - ☐ Minimal differences between some users' biometric traits
5. In a system using *Lamport hash chains* for one-time passwords, the main purpose of storing h_0 on the server is to:
- ☐ Store the user's salt for password hashing
 - ☐ Verify the entire chain by recomputing backward from the last OTP
 - ☐ Act as the user's private key for signcryption
 - ☐ Reduce the offline attack surface on the user's master password
6. When using *rate-limiting* to defend against online password guessing attacks, which of the following is a common negative side effect?
- ☐ Users experience frequent password resets
 - ☐ Users may get locked out or face increased login delays
 - ☐ Offline attackers can still freely attempt password guesses
 - ☐ The salted hashes become easier to crack
7. A system that *derives* each application password from a master password (plus the domain name) attempts to:
- ☐ Reduce the password space by reusing the same password everywhere
 - ☐ Eliminate the need for salts and peppers
 - ☐ Provide unique passwords for different sites without storing them all
 - ☐ Make offline dictionary attacks simpler to perform
8. In biometric authentication, *Failure to Enroll (FTE)* most directly impacts:
- ☐ The overall user acceptance, because legitimate users may be unable to register their biometric
 - ☐ The run-time performance of matching algorithms
 - ☐ The false accept rate (FAR)
 - ☐ Ensuring offline password cracking is infeasible
9. A 'secret salt' (pepper) is different from a normal salt because:
- ☐ It is encrypted and stored alongside the user's

- password
- ☐ It is unknown to both the server and the user
 - ☐ It is kept hidden (not stored openly in the database), forcing attackers to guess it if the database is compromised
 - ☐ It replaces hashing entirely
10. Using a passcode generator that *refreshes codes every minute* is an example of:
- ☐ A hardware token relying on an implicit time-based challenge
 - ☐ An explicit challenge-response protocol requiring a PIN entry
 - ☐ An offline password-based approach
 - ☐ A derived password approach matching domain names
-

Part B

1. **Offline vs. Online Attacks**
 - Explain the difference between *online* and *offline* password guessing attacks in **technical terms**.
 - Give **two** defense strategies specific to each type of attack.
2. **Biometric System Thresholds**
 - Discuss how a *biometric threshold* affects the False Accept Rate (FAR) and False Reject Rate (FRR).
 - Why can adjusting the threshold cause a security vs. usability trade-off?
3. **Salting vs. Peppering**
 - Compare **salting** and **peppering** in password storage.
 - Provide one example scenario illustrating the benefit of using pepper.
4. **Lamport Hash Chain**
 - Describe how a Lamport hash chain generates *one-time passwords*.
 - Why might an organization prefer Lamport chains over simply sending a user an SMS OTP?
5. **Password Composition Rules**
 - Summarize the main arguments *for* and *against* strict

- password composition rules (e.g., mandatory uppercase, symbols, length).
- Suggest an alternative policy that encourages stronger password choices without imposing composition rules.
6. **Account Recovery**
 - Compare two popular methods of account recovery (e.g., **recovery emails** vs. **secret questions**).
 - In your view, which method is more secure and why?
 7. **Rate-Limiting and Lockout Policies**
 - Present one *advantage* and one *drawback* of *strict account lockout* after a limited number of failed login attempts.
 - Suggest a policy that balances security and usability.
 8. **Graphical Password Schemes**
 - Outline **two** distinct types of graphical password schemes and their security advantages.
 - Mention one drawback for each type.
 9. **Multi-Factor Authentication (MFA)**
 - Define multi-factor authentication and explain why combining multiple factors (e.g., password + token) can improve security.
 - Provide an example of how an attacker might still bypass MFA if it's poorly implemented.
 10. **Biometrics as Non-Secrets**
 - Why do security experts say biometrics are *non-secrets*?
 - Give an example of a *realistic* attack that exploits this property and how a system might mitigate it.
-

Part C

1. **Password Space Calculation**
 - A system randomly assigns a 12-character password where each character is chosen from 62 possible symbols (A-Z, a-z, 0-9).
 - (a) Calculate the total number of possible passwords.
 - (b) If an attacker can attempt 10^8 guesses per second in an offline attack, how many **days** might it take, in the worst case, to try all possibilities?

2. **Time to Crack with Iterated Hashing**

- Suppose we have a hash function (H) that can be computed in 1 microsecond.
- If the system iterates this hash **10,000** times for each password check, how much time does **one** password verification take?
- If an attacker tries to brute-force 10^9 passwords offline and must perform these same iterations, how many *seconds* will the attacker spend in total?

3. **Biometric Error Rates**

- A facial recognition system has a **False Accept Rate (FAR)** of (0.1%) and a **False Reject Rate (FRR)** of (2%).
- (a) If 10,000 logins are attempted by legitimate users, about how many failures would you expect due to FRR?
- (b) If 10,000 login attempts are made by impostors, about how many successes (incorrect acceptances) do you expect, on average?

4. **One-Time Password Rate**

- A one-time password system (using time-based tokens) updates the code every 30 seconds.
 - If an attacker intercepts the code and tries to replay it after 45 seconds, will it succeed?
 - Discuss how this mechanism reduces replay attacks and any potential timing vulnerabilities.
-